

Beyond Similarity Search: Tenure and the Case for Structured Belief State in LLM Memory

Jeffrey Flynt
Independent Researcher
jeffrey.flynt@utexas.edu

May 2026

Abstract

Why do we need another AI to help the AI? We argue you don't. Stateless large language model sessions impose re-orientation costs on iterative, session-heavy workflows. Prior work addresses cross-session memory through retrieval-augmented approaches: store history, embed it, retrieve by semantic similarity. We argue this is the wrong abstraction. Cross-session memory is a state management problem, not a search problem. Similarity search fails for named entity resolution within bounded vocabulary contexts because beliefs about a shared technical domain are semantically proximate by construction. A single user is the simplest bounded vocabulary context; engineering teams converge on the same property through shared codebases and terminology.

We present *Tenure*, a local-first proxy that maintains a typed belief store with epistemic status, versioned supersession, and scope isolation, injecting curated context into every LLM session through precision-first retrieval rather than semantic matching. Hard scope isolation provides a structural guarantee: the right beliefs surface, and only within the boundaries the user has authorized. *Tenure*'s typed schema converts extracted facts into imperative instructions via a `why_it_matters` field, making injected beliefs directly actionable rather than raw material for the model to re-derive.

A controlled evaluation on 72 retrieval cases demonstrates the gap. Cosine similarity over dense embeddings achieves mean precision of 0.12. Alias-weighted BM25 maintains mean precision of 1.0, passing 72/72 cases versus 8/72 for cosine similarity on the same corpus. Hybrid retrieval typically solves vocabulary mismatch between disparate authors; *Tenure* eliminates this structurally: query and belief authors are the same person, and an alias enrichment flywheel continuously indexes their specific vocabulary. Under multi-turn topic drift this worsens: the vector backend produces drift scores of 0.43–0.50 on noise-critical turns where BM25 maintains 0.

Availability. Code and eval: github.com/jeffreyflynt/tenure.

1 Introduction

1.1 The Re-Orientation Tax

Every large language model session begins from a blank context window. A developer who established in a prior session that their project uses TypeScript with strict mode, Fastify for HTTP, and MongoDB with the raw driver must re-establish all of this before the model can produce useful output. Without that context, the model responds confidently in the wrong language with the wrong database and the wrong paradigm. The correction prompt is not progress; it is overhead that compounds across every session.

The same problem affects any iterative workflow. A novelist re-explains character voice. A researcher re-establishes ruled-out approaches. A consultant re-briefs the model on client decisions. The model meets the user as a stranger every time, regardless of how much was settled in prior sessions.

1.2 The Wrong Abstraction

The dominant response to this problem is retrieval-augmented memory: store prior conversation history, embed it, and retrieve semantically similar fragments at the start of new sessions. This approach, implemented by ChatGPT memory [5], Mem0 [2], Memori [1], and similar systems, treats cross-session memory as a search problem.

We argue this is the wrong abstraction for three distinct reasons. A 2025 survey formalizes agent memory as a write-manage-read loop with a three-dimensional taxonomy spanning temporal scope, representational substrate, and control policy [10]. Tenure occupies a distinct cell in that taxonomy: structured representational substrate, write-time extraction, scope-controlled injection, and compaction-based management. No system evaluated in that survey fills the same cell. The three arguments below explain why that cell is the right one for single-session LLM memory.

Memory is a state problem, not a search problem. When a user prefers explicit error returns over thrown exceptions, that preference does not need to be retrieved because it matched a query. It needs to be *active* whenever code is being generated. The retrieval framing introduces unnecessary conditionality into what should be a structural guarantee. StructMemEval [8] provides independent empirical support: simple retrieval-augmented LLMs reliably fail at tasks requiring memory organization, state tracking, and accumulated counting, even when the underlying facts are available and the retrieval budget is generous. The gap is not retrieval quality; it is the absence of an organizing structure. Both Mem0 [2] and Memori [1] arrive at the same diagnostic: as Memori states, “memory in LLM systems is not simply a storage problem, but a structuring problem.” Their solution, however, structures the write side while leaving the read side as similarity search. We argue that is an incomplete fix.

Similarity search is the wrong retrieval paradigm for bounded vocabulary stores. In any belief store where participants share a technical domain, all beliefs about that domain occupy a common semantic region. Cosine similarity captures this domain proximity but cannot discriminate within it. A query about Redis is semantically close to a belief about Redis (correct) and also semantically close to beliefs about MongoDB, TypeScript, Fastify, Kubernetes, and GitHub Actions, with cosine scores between 0.65 and 0.83. The scores reflect genuine semantic relatedness. They are measuring the wrong thing. The controlled comparison described in Section 4 makes this concrete: BM25 with alias boosting passes 60/60 static retrieval cases; vector search passes 8/60 on the same seed corpus with the same assertions. The session-level evaluation adds 12 further cases; totals

across both are cited where the full suite is relevant. The session-level evaluation (Section 6) extends this under multi-turn accumulation pressure: BM25 passes 12/12 session turns with zero drift noise; vector search passes 0/12.

Mem0 [2], Memori [1], and A-MEM [11] all identify retrieval noise as the binding constraint on cross-session memory quality. Their solutions structure the write side: extracting facts, triples, or linked notes rather than storing raw transcripts. The Mem0 ECAI 2025 evaluation [2] demonstrates the token cost consequence: full-context approaches produce median latencies of 9.87 seconds at 14 times the token cost of selective memory approaches. Memori’s LoCoMo evaluation [1] confirms the direction: structured extraction uses only 1,294 tokens per query compared to 26,031 for full-context methods. These results validate write-time extraction as correct. But all three systems then retrieve at read time using embedding similarity, leaving the precision problem unsolved.

Extraction at write time is not only cheaper. It is qualitatively better. The argument for write-time extraction is not primarily about token cost, and framing it that way understates the contribution. The model that performs extraction after a turn has the full conversational context, the user’s current intent, and the complete reasoning chain present when it writes the belief. It can record not just what was decided but why it matters for future responses. A future model receiving the injected belief gets crystallized inference, not raw material to re-derive. This timing advantage is durable in a way that token efficiency arguments are not: context windows will keep expanding, but a model reasoning from a stale transcript will never have the same fidelity as one that captured a conclusion at the moment of its formation.

Context rot is the long-run failure mode. The failure mode for persistent memory systems is not that memory is empty but that it contains stale, superseded, or contradictory facts that silently influence responses. Memori explicitly names this condition: “context rot, in which relevant information is present but not effectively used” [1]. A memory architecture that does not model supersession will eventually produce confident responses grounded in outdated context.

1.3 Contributions

1. **A formal belief schema** (Section 3) with five belief types, four epistemic statuses, typed provenance, and a `why_it_matters` field that converts extracted facts into pre-computed action instructions.
2. **A precision-first retrieval design** (Section 4) with empirical justification: BM25 with alias boosting passes 72/72 retrieval cases (60 static, 12 session-level); cosine similarity over dense embeddings passes 8/72 on the same corpus.
3. **A compaction architecture** (Section 5) that prevents noise floor accumulation, including an alias enrichment flywheel and a counter-signal property demonstrated in the evaluation data.
4. **A 72-case retrieval evaluation suite** (Section 6) covering alias resolution, scope disambiguation, supersession chain exclusion, fuzzy matching, cross-user isolation, budget eviction, ranking stability, and session-level noise isolation under multi-turn topic drift, published as a reusable benchmark.

2 Related Work

2.1 The Incomplete Fix in Recent Memory Systems

Tenure structures both sides: typed beliefs with epistemic status and scope at write time, and alias-weighted BM25 with hard scope isolation at read time.

Mem0 [2] establishes write-time extraction as an architectural commitment shared with Tenure: facts are extracted at write time rather than retrieved from raw transcripts. Mem0 stores these facts as natural language strings in a flat graph with relationship edges and operates as a hosted service where both conversation content and extracted memory transit to a provider. Tenure’s contributions over that baseline are the typed schema with epistemic status, the scope system as a structural guarantee, the supersession mechanic, the compaction architecture, and a local-first deployment model.

Memori [1] is the closest published system to Tenure’s architectural position. Both reject raw transcript injection, treat memory as a structuring problem, and extract structured representations at write time. Memori’s Advanced Augmentation pipeline converts dialogue into semantic triples (subject-predicate-object) linked to conversation summaries, achieving 81.95% accuracy on LoCoMo using only 1,294 tokens per query.

The architectural contrast is the most instructive point of comparison and goes beyond implementation choice to a fundamental representational difference. Memori’s semantic triples are declarative: a triple records a fact. Tenure’s `why_it_matters` field is imperative: it records what a future model should do because of that fact. “TypeScript with strict mode” describes a belief; “shapes all code examples toward TypeScript with strict mode and no implicit any” is an instruction the model can act on without further inference. This distinction is the real answer to why structured extraction produces better responses than triple retrieval. The extracting model has the full conversational context, the user’s current intent, and the complete reasoning chain present when it writes `why_it_matters`. A future model receiving the injected belief gets crystallized inference rather than raw material to re-derive. This is a genuine architectural difference, not a tuning parameter.

Memori’s own evaluation documents the temporal reasoning gap that the Tenure session evaluation cases expose: temporal scores (80.37%) trail single-hop reasoning (87.87%) because “isolated semantic triples capture static facts but often miss the temporal context needed to identify changes in user states or preferences across sessions” [1].

A-MEM [11] treats memory as a self-organizing network of Zettelkasten-inspired notes where new memories trigger updates to the contextual representations of existing ones. This evolution mechanism inherits the context rot problem by design: there is no supersession chain, no audit log, and no structural guarantee that outdated context is retired.

2.2 Structured Memory Evaluation

StructMemEval [8] evaluates an agent’s ability to organize long-term memory rather than merely retrieve facts. Its central finding maps directly onto Tenure’s core argument: “simple retrieval-augmented LLMs struggle with these tasks, whereas memory agents can reliably solve them if prompted how to organize their memory.” The benchmark isolates tasks requiring state tracking, hierarchical organization, and accumulated counting, and demonstrates that retrieval-augmented systems fail at these tasks even when the underlying facts are available. Tenure’s belief schema is a practical implementation of the organizing structure StructMemEval formalizes.

Critically, StructMemEval documents the hallucination accumulation failure mode that compaction is designed to address: spurious memory hallucinations become more frequent as the LLM performs hundreds of consecutive memory updates [8]. The vector evaluation results suggest the mechanism behind this: when every query retrieves a full corpus of semantically proximate beliefs, each turn gives the model more noise to potentially reinforce. Structured extraction feeding into precise retrieval is the architectural answer at both write time and read time.

H-Mem [12] proposes a hybrid multi-dimensional memory architecture with temporal and semantic parallel trees. Its intersective search mechanism uses a second discriminating filter after initial retrieval rather than relying on a single retrieval signal. H-Mem’s 8.4-point improvement over single-axis retrieval on LoCoMo provides independent empirical support for structural post-search filtering. H-Mem’s human evaluation also documents a relevant ceiling: human evaluators outperform H-Mem on temporal J-Score (78.19 versus 57.63), reflecting human superiority at temporal reasoning over long conversations [12].

2.3 Dual-Route Retrieval and Structural Discrimination

Mnemis [9] achieves state-of-the-art performance on LoCoMo (93.9) and LongMemEval-S (91.6) using dual-route retrieval that combines System-1 similarity search with a System-2 Global Selection mechanism over a hierarchical graph. The hierarchical graph enables top-down traversal over semantic categories, retrieving entities that are structurally relevant but semantically distant from the query. This is an architecturally adjacent observation: using structural signals to discriminate within a semantically proximate retrieval set is the same problem Tenure’s scope isolation and alias weighting address, approached from the read side rather than the write side.

The distinctions are at the level of architectural commitment rather than retrieval mechanism. Mnemis still retrieves and re-ranks rather than maintaining typed state: it has no epistemic status, no supersession chain, and no scope isolation as a hard structural guarantee. A belief that was true last month and is now superseded would remain in Mnemis’s graph unless manually removed, because the graph models relationships, not the lifecycle of the facts those relationships encode. Mnemis’s strength (93.9 on LoCoMo) is on a benchmark that measures conversational recall quality. LoCoMo does not test scope isolation, supersession correctness, counter-signal retrieval, or accumulation-induced precision degradation under multi-turn topic drift. The 72-case evaluation suite described in Section 6 covers properties those benchmarks do not measure, not competing metrics on the same benchmark. Mnemis’s hierarchical graph is a read-time structure that improves retrieval over accumulated memory. Tenure’s alias enrichment flywheel and compaction are write-time structures that increase precision over time. The architectural commitments are at different points in the pipeline and address different failure modes.

2.4 Named Entity Resolution and BM25

The choice of BM25 [7] over embedding similarity is grounded in the named entity resolution literature [4]. For single-user knowledge stores where the user coined the terminology and aliases were authored to match expected query surfaces, BM25 with alias boosting provides higher precision than cosine similarity. Embedding similarity retrieves what is semantically related; alias-weighted BM25 retrieves what the user named. These are different queries, and in a single-user persistent memory context, the latter is more often correct. Memori’s hybrid retrieval design [1] reflects a compatible observation from a multi-user setting: keyword matching is necessary even when embeddings are available. Tenure’s precision-first argument goes further: wherever aliases are

authored to match expected query surfaces within a bounded vocabulary context, BM25 alone with alias boosting provides sufficient precision and avoids the false positive retrievals that semantic similarity introduces. The single-user case is the cleanest demonstration; the property holds for any context where vocabulary has converged, including engineering teams with shared codebases.

3 The Belief Architecture

3.1 Overview

A *belief* is the atomic unit of persistent context. The typed belief schema with epistemic status is a practical descendant of the Belief layer in the Belief-Desire-Intention (BDI) agent architecture [6]. BDI formalizes beliefs as the agent’s informational state (what it holds to be true about the world), distinct from its goal state (Desires) and committed action plans (Intentions). Tenure inherits this representational commitment: the belief store is the informational substrate the model reasons from, not a search index it queries. The departure from BDI is scope: Tenure does not model Desires or Intentions, because the LLM session provides those dynamically. What persists across sessions is the belief state alone.

Beliefs are not conversation fragments; they are extracted conclusions, typed, scoped, and versioned, that can be injected into future sessions without requiring the model to re-derive them from raw history. The belief store controls all factual state; the model renders surface responses from it. StructMemEval [8] provides independent empirical motivation: tasks requiring state tracking and hierarchical organization are reliably solved by structured memory agents but not by retrieval-augmented systems.

The contrast with Memori’s semantic triples [1] is precise: a triple records a fact; a belief records a fact, its epistemic status, its scope, its provenance, and its supersession history, along with an imperative instruction for how a future model should act on it. The additional structure is the mechanism that prevents context rot rather than merely describing it.

Definition 1 (Belief). *A belief b is a 14-field tuple with the following typed components:*

- $type \in \{\text{preference}, \text{decision}, \text{entity}, \text{open_question}, \text{relation}\}$
- $subtype \in \{\text{expertise}, \text{style}, \text{null}\}$
- $epistemic_status \in \{\text{active}, \text{inferred}, \text{exploratory}, \text{superseded}\}$
- $scope \subseteq \mathcal{S}$, the set of valid scope labels
- $confidence \in [0, 1]$
- $superseded_by \in \{id'\} \cup \{\text{null}\}$

A belief is *active* if `superseded_by` = `null` and `resolved_at` = `null`.

3.2 The `why_it_matters` Field as Pre-Computed Action

The `why_it_matters` field is the architectural feature that most sharply distinguishes a belief from a semantic triple or a flat extracted fact. Every belief requires a `why_it_matters` value: one sentence on what future responses this fact shapes. The extraction instruction enforces this as a quality gate: if the extracting model cannot write `why_it_matters` clearly, the belief is not extracted.

The distinction is between declarative and imperative representation. “Uses TypeScript with strict mode” is a fact. “Shapes all code examples toward TypeScript with strict mode and no implicit any” is an instruction. A model receiving the fact must still infer how to apply it to the current response. A model receiving the instruction can act on it directly without additional inference work. The imperative form is what makes structured extraction qualitatively better than transcript injection, not merely token-efficient.

This field is why extraction timing matters independently of token cost. The extracting model writes `why_it_matters` when it has the full conversational context, the user’s current intent, and the complete reasoning chain present. The crystallized instruction it produces will be correct and actionable for any future query on the topic. A model receiving a raw transcript must reconstruct this inference from older, decontextualized material. The advantage is not cheaper context injection. It is that the right inference was captured at the moment of its formation.

The persona prelude operates at a layer above individual beliefs. It is generated as natural language prose from the accumulated belief state and injected unconditionally on every session, independent of retrieval. Where individual beliefs provide specific, query-matched instructions, the persona prelude provides standing behavioral instruction: how the user wants to be communicated with, what working style they prefer, how the model should behave when context is ambiguous. A persona prelude that instructs the model to ask clarifying questions before generating changes the model’s first move on every underspecified query, not because a relevant belief was retrieved, but because the behavioral instruction is structurally present. This is the mechanism by which Tenure addresses the cold-start problem: even a partially completed onboarding produces enough persona signal to replace confident generation into unknown context with calibrated uncertainty.

3.3 Belief Types

Preferences represent how the user works and communicates. Scoped to `user:universal` when domain-independent, or to a specific domain or project. Preferences shape model behavior without requiring query matching.

Decisions represent resolved commitments including the rejected alternatives, preventing re-litigating settled questions in future sessions.

Entities represent named things with alias resolution. A belief about Kubernetes carries aliases `k8s` and `kube` so that queries using either short form retrieve the correct entity.

Open questions represent unresolved matters that future sessions should surface rather than answer.

Expertise beliefs (a subtype of preference) represent depth calibration. An expertise belief at the deep level in `javascript/react` licenses the model to skip fundamentals and engage on trade-offs without explanation.

3.4 Epistemic Status

Epistemic status is a first-class field rather than a derived property of confidence. The **active** versus **inferred** distinction determines injection behavior and the threshold for promotion across sessions. The **active** versus **superseded** distinction is the architectural mechanism that prevents context rot.

Superseded beliefs are retained in the store for audit purposes but are never injected into any session. A system that deletes stale beliefs cannot distinguish between “we never had this belief” and “we had this belief and moved past it.” This is the structural property that neither Memori’s semantic triples [1] nor Mem0’s flat memory graph [2] provide.

Promotion from **inferred** to **active** requires two independent conditions: a minimum reinforcement count and a minimum age. The age condition ensures that reinforcements represent independent observations separated in time rather than repeated signals from the same causal chain within a single session.

3.5 Scope

Scope is a list of labels assigned at extraction time. **Universal scope** (`user:universal`) marks beliefs injected across all sessions. **Domain scope** (`domain:code`, `domain:writing`) marks beliefs applying only within a specific domain. **Project scope** (`project:<slug>`) marks beliefs applying only to a specific project.

Scope assignment operates at two levels. The scope detector infers scope automatically from the first message of each session, matching against existing belief scopes or proposing new ones. Users may override this at any time during an active session with an explicit scope command (`!scope domain:writing`), which shifts the belief context immediately for all subsequent turns without starting a new session. Beliefs from the previous scope stop surfacing; beliefs from the new scope surface immediately. Session history remains intact while the belief context pivots cleanly.

An administrative setting restricts scope assignment to explicit user commands only, disabling automatic inference entirely. In this mode the system provides a deterministic contract: beliefs surface if and only if the user has declared an authorizing scope. No scope declaration means only `user:universal` beliefs surface. Domain and project beliefs are structurally absent rather than probabilistically suppressed, directly analogous to strict type checking: the guarantee is only meaningful if it is unconditional.

3.6 Injection Format: The Lean Projection Principle

The context injection format applies a deliberate principle: inject only fields that change model behavior, not all fields that describe the belief. Epistemic status is only surfaced when it is not the default (non-**active** beliefs carry a hedging signal). Confidence is only surfaced when it should affect model weighting (below 0.65). The type field is only included when it carries instruction value (decisions and open questions), not when it is self-evident from content.

This is the difference between a principled token budget and simply being frugal. The epistemic status and confidence thresholds are concrete instantiations of the principle: fields that do not change how the model responds to a given belief should not consume tokens. The persona prelude is generated as natural language prose rather than structured JSON for the same reason: it represents standing behavioral instruction that should be internalized as ambient context, while the typed belief tiers are structured JSON because each belief is a discrete fact to be applied conditionally based on its type, epistemic status, and relevance to the current query.

3.7 Provenance and Change Log

Every belief carries a provenance record identifying the session, turn, timestamp, and source model from which it was extracted. Every mutation appends an entry to an append-only change log. This

structure makes every belief auditable and provides the data necessary to replay belief history if a belief needs to be corrected or rolled back.

4 Retrieval Design

4.1 The Precision-First Argument

The field has correctly diagnosed the disease and then prescribed more of the pathogen. Write-time extraction is right; similarity search at read time reintroduces the noise that structured extraction was designed to eliminate. The precision-first argument is the architectural answer to that gap: wherever aliases are authored to match expected query surfaces, term matching over indexed aliases provides higher precision than semantic similarity by construction.

The simplest case is a single user who coined the terminology: if they named their Kubernetes belief with canonical name `kubernetes` and aliases `k8s` and `kube`, then a query containing `k8s` should retrieve that belief with high precision regardless of semantic distance. There is no ambiguity to resolve: the authored terminology is the ground truth. The same property holds for an engineering team of twenty whose shared codebase, runbooks, and tickets have converged on consistent vocabulary for the same entities.

Proposition 1 (Alias retrieval dominance for bounded vocabulary stores). *In a belief store where aliases are authored to match expected query surfaces within a bounded vocabulary context (whether single-user or shared-team), BM25 retrieval over the alias field with a high boost weight provides higher precision than embedding cosine similarity for named entity queries, because aliases are exact or near-exact matches to expected query terms by construction.*

The proposition is supported rather than formally derived: the structural mechanism is documented in Section 4 (index-search analyzer asymmetry and boost calibration) and the empirical validation is in Section 6.

This is not an assumption about system design; it is a documented property of individual language production. Corpus-based idiolect research establishes that single speakers maintain stable, distinctive lexical choices across production contexts, with idiolectal patterns consistent over periods of one to two years and rooted in core linguistic constructions rather than peripheral idiosyncrasies [13]. Lexical priming theory formalizes the mechanism [14]: words become entrained through use, and speakers reliably return to the same lexical choices in the same topical contexts [15]. A single-user belief store is precisely the setting where these properties are strongest—the query author and the belief author are the same person, and the vocabulary they use to query is the vocabulary they used when the beliefs were authored.

Memori’s hybrid retrieval design [1], which combines cosine similarity with BM25 over triple content, reflects a compatible observation from a multi-user setting: keyword matching is necessary even when embeddings are available. Tenure’s precision-first argument goes further: for any store where aliases are authored to match expected query surfaces within a bounded vocabulary context, BM25 alone with alias boosting provides sufficient precision and avoids the false positive retrievals that semantic similarity introduces. The evaluation data demonstrates this for a single-user corpus; the mechanism extends to team-level stores where vocabulary convergence produces the same structural property.

4.2 Empirical Comparison: BM25 versus Vector Search

To validate the precision-first argument, the full 60-case static evaluation suite was run against two retrieval backends using identical seed corpora, identical test harnesses, and identical assertion logic. A 12-case session evaluation suite was subsequently run against the same two backends to validate precision stability under multi-turn accumulation pressure.

The **BM25 backend** uses alias-weighted text search with structural scope isolation. The **vector backend** uses nomic-embed-text embeddings (768 dimensions) with cosine similarity, with pre-filters on user ID and superseded status applied before scoring.

Table 1: The score distribution for a representative query; the mechanism is structural and independent of threshold or model choice.

Metric	BM25 (alias-weighted)	Vector (cosine)
Static cases passed	60/60	8/60
Session cases passed	12/12	0/12
Total cases passed	72/72	8/72
Mean precision (<code>shouldOnlyInclude</code> cases)	1.0	0.12
Mean recall (<code>shouldOnlyInclude</code> cases)	1.0	1.0
Session drift score (noise-critical turns)	0.0	0.43–0.50
Redis scope disambiguation	Correct	Failed
Score boundary (weak query no match)	Correct	Failed
Ranking stability (IDF weighting)	Correct	Failed

The mechanism is structural, not a matter of threshold or model choice. All beliefs about one developer’s stack are genuinely semantically proximate: Redis, TypeScript, Fastify, MongoDB, Kubernetes, and GitHub Actions all belong to the same infrastructure and development domain. No threshold or re-ranking layer resolves this because the problem is the retrieval paradigm itself, not the implementation.

A larger embedding model would distribute scores differently but cannot eliminate genuine semantic proximity within a domain-specific corpus: the beliefs *are* semantically related, and the 0.132 cosine spread across twelve beliefs is a measurement of real proximity, not an artifact. Resolving it requires a retrieval signal orthogonal to semantic similarity.

This result explains a pattern in the recent literature. Memori documents a temporal reasoning gap and attributes it to isolated triples missing temporal context [1]. StructMemEval documents that retrieval-augmented systems fail at state tracking tasks even when facts are retrievable [8]. The vector evaluation data suggests a common mechanism: the correct fact is retrievable but not *exclusively* retrieved, and noise from semantically adjacent retrievals degrades downstream reasoning. A system that retrieves one correct belief and zero noise beliefs provides unambiguous signal. A system that retrieves one correct belief and nine noise beliefs asks the model to discriminate relevance at generation time—which is precisely the cognitive overhead that structured extraction was designed to eliminate at write time. The clause-level score attribution emitted by the evaluation harness makes this mechanism inspectable rather than opaque: every retrieval decision records which alias matched, on which index path, at what weight. No equivalent transparency is possible over a cosine similarity score.

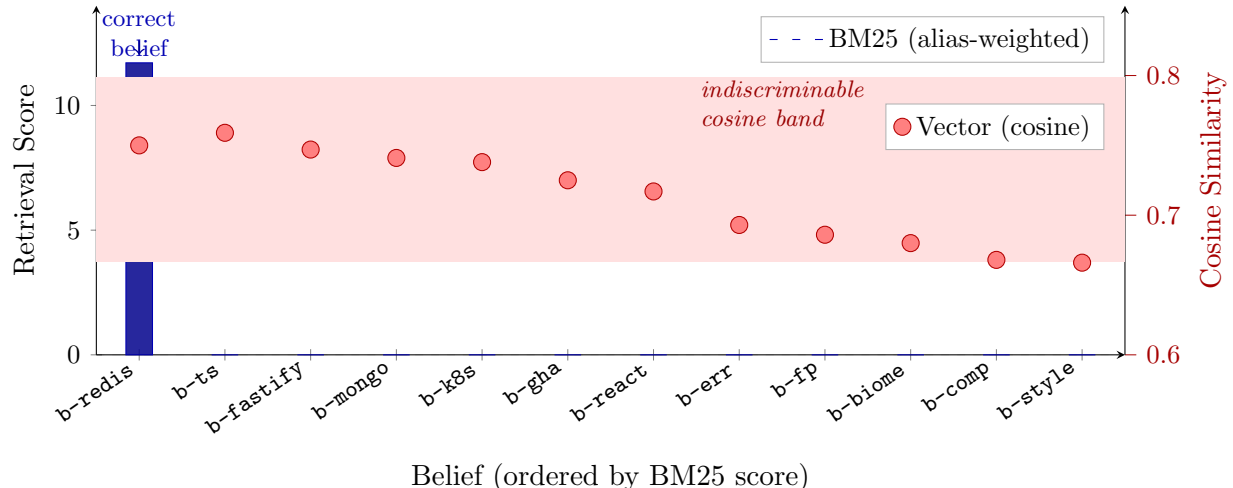


Figure 1: BM25 returns exactly one belief (**b-redis-code**, score 11.71); all others score zero. Vector search returns all twelve beliefs in a 0.132-wide cosine band (0.667–0.799), with the correct belief ranked second behind **b-ts-pref** at 0.759, recall is identical, the gap is entirely precision. Query: “What are we using Redis for?” against the 30-belief seed corpus spanning two domain scopes and one secondary-user isolation fixture. The shaded region marks the indiscriminable cosine band.

4.3 Search Architecture and Index Engineering

The Inverted Retrieval Problem Standard information retrieval assumes short queries against long documents. BM25 is calibrated for this setting: a three-term query matched against a thousand-word document, where IDF weighting rewards terms that are rare across the document corpus and term frequency in the document body carries discriminative signal.

Belief retrieval inverts this relationship entirely. The query is potentially hundreds of tokens of natural language: a rambling conversational turn, a compound engineering question, a verbose problem description. The documents are short: a canonical name of one to three tokens (**redis_cache**, **github_actions**) and an alias list of three to five short terms. Standard BM25 applied naively to this inverted setting fails predictably. The query’s term frequency becomes noise rather than signal. IDF weights are computed over a tiny corpus of short technical identifiers rather than a large prose corpus. The standard analyzer’s stemming and stopwords behavior is calibrated for natural language prose rather than proper nouns.

The evaluation data confirms what this analysis predicts. The 400-character Kubernetes rolling deployment query succeeds not because BM25 handles verbose queries gracefully by default, but because the index is specifically engineered for the inverted problem. The **canonical_name_analyzer** maps **redis_cache** to discrete matchable tokens. The shingle analyzer produces phrase-level match surfaces from short alias lists. The 14x boost on canonical name and alias phrase matches recalibrates the score distribution so that a single precise match on a short field outweighs the accumulated term frequency noise from a long query. The search-side analyzer asymmetry ensures the query is tokenized conventionally while the index side is structured for precision against short fields.

Most practitioners abandoning BM25 for this domain would do so at the first step: short documents matching long queries, before reaching the engineering that makes it work. The result is not that

BM25 is the right retrieval paradigm by default. It is that BM25 with domain-specific index engineering for the inverted retrieval problem outperforms semantic similarity on the precision metric that determines whether injected context helps or harms the downstream model.

Index-Search Analyzer Asymmetry Atlas Search permits independent analyzer configuration for indexing and querying on the same field, a capability most BM25 implementations do not expose. Tenure exploits this asymmetry at two levels.

The `canonical_name` field is indexed with a custom analyzer that maps underscores to spaces and uses a regex-capture tokenizer, so `redis.cache` enters the index as two tokens: `redis` and `cache`. The search side uses `lucene.standard`, which tokenizes the query conventionally. A query containing `redis` produces the token `redis`, which matches the indexed token from `redis.cache`. Without the asymmetry, a query would need to contain the literal string `redis.cache` to match the canonical name field. Every canonical name in the belief store uses snake_case by convention, so this would make canonical name matching fail on essentially every natural language query.

The `aliases` field is indexed with the same canonical name analyzer but searched with `aliases.light`: whitespace tokenization, lowercase, and English possessive stripping, with no stemming. Stemming is deliberately excluded from the search side because alias terms are proper nouns and technical identifiers. A stemmer would collapse `kubernetes` and `kube` toward a common stem, potentially conflating aliases that should remain discrete match surfaces. The asymmetry preserves exact alias matching on the query side while the index side handles the structural normalization.

The shingle paths add a third asymmetry: the index stores bigram pairs from the canonical name analyzer output, while the search side runs `canonical_name.search_analyzer`, which applies whitespace tokenization and then shingle generation, producing the bigram `base class` at query time to match the pre-indexed shingle. This is what produces the score separation visible in the evaluation data: the query “should I use a base class hierarchy” generates the shingle `base class`, which matches the pre-indexed shingle on `b-composition-inheritance` at the 14x boost weight, producing a score of 29.75 on turn 8 of the session evaluation. A single-token match on `base` alone would score in the 3–4 range.

This three-level asymmetry is the mechanism that makes BM25 work on natural language queries against a snake_case belief store with multi-word aliases. The result is not a property of BM25 in general; it is a property of index-search asymmetry applied deliberately to exploit the structural characteristics of the belief schema.

Stage 1: Query preparation. The raw user message passes through a noise-stripping pipeline that removes code blocks, file references, stack traces, URLs, and markup. The database’s standard analyzer handles tokenization, lowercasing, and IDF-weighted scoring. Stop words and filler terms score near-zero naturally due to high document frequency.

Stage 2: Alias-weighted text search. A compound query runs over two paths: canonical name with a high boost weight and aliases with a secondary boost weight. Fuzzy matching with `maxEdits: 1` and `prefixLength: 2` applies to both paths. Hard filters on user ID, resolved status, and superseded status ensure that resolved and superseded beliefs never appear as candidates.

Stage 3: Scope filter as hard discriminator. After search scoring, a hard scope filter is applied as a post-search match stage. This two-stage design is what resolves the Redis disambiguation: both Redis beliefs score identically under BM25 because they share the alias; scope determines which one is returned.

The prefix guard (`prefixLength: 2`) blocks transitive matches that edit distance alone would permit: `mango` (edit distance 1 from `Mongo`) is blocked because the prefix `ma` does not match `mo`. This is an explicit precision trade-off documented in the evaluation suite.

The pinned facts tier. Pinned beliefs and all active preferences are retrieved unconditionally, independent of text search scoring. A decision that the project uses MongoDB does not need to be query-matched to be relevant to a data layer question; it is relevant by type.

Type routing. Open questions are retrieved by a separate path that returns only pinned open questions for the active scope. They are never returned by text search. Type routing takes precedence over both pin status and retrieval score.

4.4 Scope and Teams

The precision-first argument is sometimes characterized as limited to single-user stores on the grounds that enterprise contexts require semantic fuzziness to handle vocabulary variance across large teams. This conflates two distinct retrieval problems. Semantic fuzziness addresses vocabulary uncertainty: when query author and belief author are unknown to each other, embedding similarity is the correct signal. Within any context where vocabulary has converged and aliases are authored to match expected query surfaces, alias-weighted BM25 produces higher precision for the same structural reason established in Section 4.

The vector precision problem does not merely persist at team scale; it worsens. A team of twenty developers working on the same infrastructure produces a denser cluster of semantically similar beliefs: more PRs, more architectural decision records, more deployment docs, all occupying the same semantic region. A vector search for “auth” returns fifty similar results spanning every service that touches authentication. BM25 with alias boosting returns the specific “Auth Service” belief the team has codified, because the alias matches the query term and scope isolates the result. The session evaluation data demonstrates this mechanism in miniature: with only twelve non-pinned beliefs, the vector backend’s drift score climbs to 0.50 on noise-critical turns as accumulated semantic mass from diverse topics creates broader overlap. With two hundred beliefs from a twenty-person team, the cosine scores would compress further into an indiscriminable band. BM25 precision is invariant to corpus density because its retrieval signal is term presence in indexed fields, not semantic proximity to accumulated context.

5 Extraction, Compaction, and the Tier Gate

5.1 Extraction Architecture

Belief extraction runs asynchronously after the response is returned to the client, never blocking the request path. Every LLM response includes a structured sidecar block appended after the visible response content containing an extraction result: new beliefs, belief updates, entity updates, alias candidates, resolved open questions, new open questions, and style signals. The sidecar is stripped before the response is returned to the client.

This design enforces a strict separation between proposal and commitment: the model generates a structured proposal via the sidecar, but a validator, merger, and scope system determine what actually enters the belief store. The model never directly writes a belief.

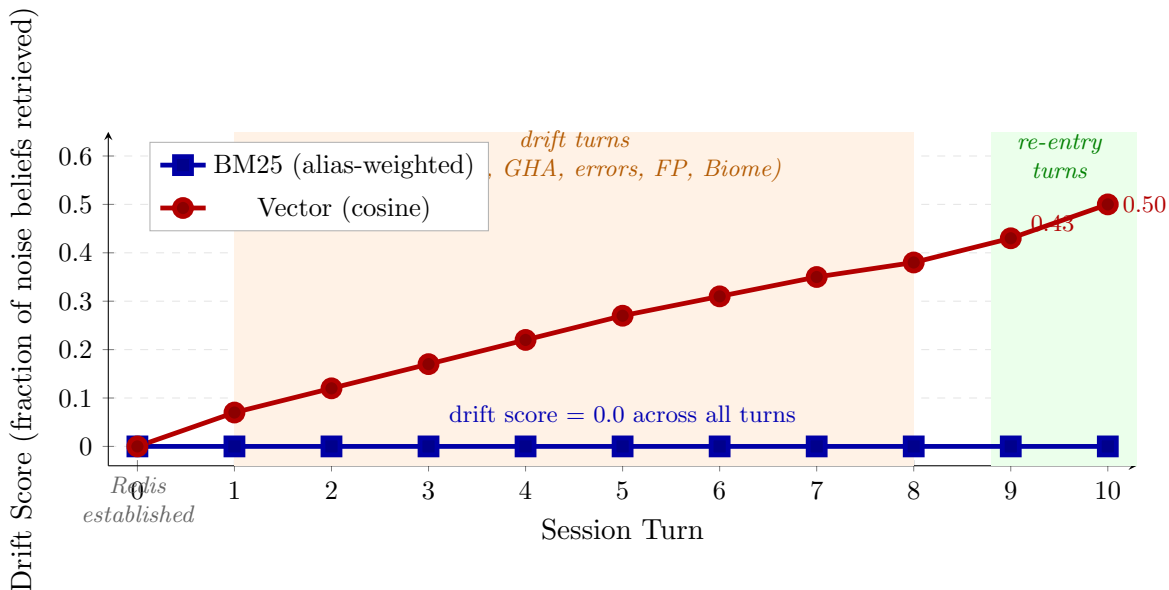


Figure 2: The turn 10 spike reflects the cross-session formative case: a subsequent session queries `allkeys-lru`, an alias added to `b-redis-code` by the extraction worker after turn 9, retrieving the correct belief at score 10.26, but the vector backend simultaneously retrieves every other belief in the corpus, producing a drift score of 0.50.

5.2 The Extraction Tier Gate

Reliable structured belief extraction requires frontier model output consistency. The system enforces a model tier gate at the extraction boundary: only models meeting a minimum structured output reliability threshold are permitted to drive extraction. The current gate covers a defined set of frontier models documented in the project README [3]; the README is the authoritative and maintained reference for which models clear the threshold as the landscape evolves.

The gate is a design principle rather than a limitation. An architecture whose correctness depends on extraction quality cannot treat that dependency as optional. The consequence is explicit and correct: sessions routed through models below the threshold receive responses but no belief extraction. The belief store does not degrade with malformed inputs. StructMemEval’s finding [8] that LLMs capable of abstract algorithmic reasoning frequently fail to apply the same organizational patterns to their own memory operations is the independent empirical grounding for this design choice.

5.3 Conflict Resolution

The merger implements a four-branch policy: insert (no matching belief, confidence meets threshold), reinforce (matching belief with same content, reinforcement count incremented), flag conflict (content differs within confidence margin, queued for user review), and skip (content differs without sufficient confidence margin). Explicit update signals handle reinforcement, contradiction, and supersession. Supersession creates a new belief and marks the old one with a `superseded_by` pointer, preserving the full chain for audit.

5.4 Observability as a Structural Property

A typed, human-readable belief store makes memory state observable in a way that vector stores cannot be. When an extracted belief contradicts an active one, the system surfaces the conflict as an explicit, reviewable event rather than silently absorbing it. The user receives a notification rather than an audit problem. Every belief carries a provenance record and an append-only change log, so the history of how a belief arrived at its current state is inspectable without reconstructing it from session transcripts. The dashboard makes every belief visible, editable, and correctable with the cognitive load of reviewing a short notification rather than auditing an opaque embedding index.

No equivalent exists in a vector store. A contradiction between two embeddings is invisible because embeddings do not have content in a form that can be compared, flagged, or reviewed. The store absorbs both and retrieves both. The model sorts out the contradiction at generation time, which is the cognitive overhead that structured extraction was designed to eliminate at write time. The vector evaluation results make this concrete: the system returned ten beliefs per query including contradictory and stale signals, leaving disambiguation to the model on every turn. The precision gap (1.0 versus 0.12) is not only a retrieval quality difference. It is the difference between a system that manages belief state and a system that searches over stored text.

5.5 Compaction

The belief store grows monotonically as new beliefs are extracted. Without compaction, the store accumulates duplicate, overlapping, and redundant beliefs that degrade retrieval precision and inflate context size. StructMemEval [8] documents this empirically: hallucinations of spurious memories become more frequent as the LLM performs hundreds of consecutive memory updates. A-MEM’s scaling analysis [11] provides a quantitative baseline: retrieval time at 1,000,000 entries is 12 times higher than at 1,000 entries.

Three distinct compaction paths run asynchronously, triggered by belief count thresholds per scope. **Preference deduplication** uses a conservative prompt that merges only when highly confident two beliefs express the same fact. **Entity deduplication** follows the same structural mechanics at a higher threshold. **Expertise synthesis** produces structured assessments per domain from accumulated expertise signals, respecting a subdomain hierarchy where leaf domains are assessed independently and parent rollups are generated only when multiple subdomains collectively justify one. The depth calibration rubric has four levels (learning, working, deep, expert) with distinct injection semantics. Expertise synthesis invalidates the persona cache, triggering regeneration to reflect updated depth assessments.

When two or more inferred beliefs are collapsed by compaction, their combined reinforcement count is checked against a promotion threshold that is lower than the standard merger threshold. Two independently extracted inferred beliefs collapsing into one represents qualitatively stronger evidence than repeated reinforcement of a single belief, because independent extraction events are causally independent.

5.6 Alias Enrichment as a Precision Flywheel

BM25 precision for a belief store is highest when the alias set of each belief matches the vocabulary the user actually uses in queries. That vocabulary is not fully known at extraction time. Tenure captures it continuously through three mechanisms operating at different levels.

At the turn level, the extraction model emits `entity_update` signals when it observes a surface

form for an existing belief that is not yet indexed as an alias. The merger applies these signals to extend the belief’s indexed terms with each new observed form. Aliases are normalized to lowercase on write and deduplicated automatically.

At the compaction level, retired belief canonical names are preserved as aliases in merged beliefs. When two beliefs merge into a canonical form, every name that previously retrieved either belief continues to retrieve the merged one. Vocabulary accumulated before a compaction event is never lost.

Alias overlap across beliefs does not produce ambiguous retrievals because scope isolation is applied as a hard post-search discriminator. Two beliefs sharing an alias in different scopes (e.g., `redis` appearing in both `domain:code` and `domain:writng`) are never co-retrieved: the active scope selects exactly one. Within a single scope, alias sets are kept intentionally narrow at extraction (3–5 terms) and grow through observed usage to a hard ceiling of 25. The structural ceiling combined with scope filtering bounds the collision surface regardless of how many beliefs accumulate over time.

The third mechanism is the counter-signal property. Aliases also include the canonical names of superseded beliefs and the names of things the user has explicitly rejected or moved away from. If the user uses Fastify, the alias set for the Fastify preference includes `express`, so a query about Express surfaces the Fastify preference as a counter-signal rather than returning silence. This is not a search feature. It is a memory architecture property: the system surfaces what the user actually uses when a query references what they rejected. The evaluation data demonstrates this working: the query “suggest a good Mongoose schema for my user model” retrieves `b-mongo-raw-driver` via the `moongoose` alias with score 9.29. This property is absent from every system surveyed. Mem0 stores facts as natural language strings with no counter-signal mechanism, Memori’s semantic triples record what is true without indexing what was rejected, and A-MEM’s evolution mechanism updates existing notes but does not preserve superseded alternatives as query paths.

The session evaluation provides end-to-end demonstration of the alias enrichment flywheel in action. In the cross-session case, after the extraction worker enriches `b-redis-code` with aliases `allkeys-lru` and `maxmemory` from the formative turn, a subsequent query referencing those terms retrieves the correct belief with a combined BM25 score of 6.73 from the two alias matches. The vector backend also retrieves the belief (score 0.919) but simultaneously retrieves every other belief in the corpus. The flywheel produces precision; embedding similarity produces recall without discrimination.

The consequence is a precision flywheel: the more sessions accumulate, the more surface forms and counter-signals are indexed, and the more precisely BM25 resolves the vocabulary that is actually used. Consider a user whose cat is named Pudding. The belief is authored with the alias `pudding`. After sessions where the cat is referred to as “my cat,” “the cat,” and “my baby,” all four forms are indexed and resolve to the same belief. The belief becomes more findable with each session, not less.

5.7 Session Compaction and the Unified Principle

The compaction architecture applies to the belief store, but the same principle operates at the session history level simultaneously. The session history compaction system collapses turns by type: acknowledgment turns are collapsed entirely, turns whose beliefs are all active and not commitments are collapsed, and completed off-topic turns are collapsed to condensed markers. The `renderHistory` function produces markers like “4 earlier turns condensed” rather than injecting raw history.

This is not incidental engineering parallelism. Both compaction paths instantiate the same design principle at different levels of the information lifecycle: inject the minimum representation that preserves behavioral signal. At the belief level, overlapping facts are merged into canonical form. At the session level, semantically redundant turns are collapsed into markers. In both cases, the model receives structured signal rather than raw material it has to process.

6 Evaluation

6.1 Evaluation Suite Design

The evaluation suite is a set of 72 test cases covering retrieval properties of the belief architecture: 60 static single-query cases and 12 session-level multi-turn cases. Each static case specifies a user identifier, a domain scope, a query string, an optional budget override, and expected properties of the retrieval result at four tiers: pinned facts, relevant beliefs, open questions, and persona prelude. Session cases specify a multi-turn conversation with per-turn retrieval assertions and noise-check constraints.

The evaluation runs against a local Atlas Search instance with a fixed seed corpus of 30 beliefs: one primary user’s beliefs spanning two domain scopes (`domain:code` and `domain:writing`) and a three-hop supersession chain, plus a single secondary-user fixture included exclusively to validate cross-user isolation.

No existing benchmark evaluates the combination of alias resolution, scope disambiguation, supersession chain exclusion, and cross-user isolation for single-user persistent memory systems. StructMemEval [8] evaluates memory organization capabilities but focuses on tree structures, state tracking, and counting tasks. The benchmarks used in the Mem0 ECAI evaluation [2] and the Memori LoCoMo evaluation [1] cover conversational recall quality at scale; neither tests scope isolation, supersession correctness, counter-signal retrieval, or budget behavior under load.

The 72-case suite is deliberately scoped to correctness properties for which corpus size is irrelevant. Scope isolation either holds or it does not; supersession chain exclusion either holds or it does not; counter-signal retrieval either works or it does not. These are binary structural guarantees, not performance curves. A sort algorithm is not validated by running it on a million items but by exercising the cases where it should and should not fire; scale testing is a separate question about a separate property. Conflating corpus size with evaluation rigor mistakes load testing for correctness testing. The 72-case suite fills the gap those benchmarks leave: the structural properties that determine whether a memory system remains correct over time.

6.2 Case Categories

Alias resolution (18 cases). Short-form aliases (`k8s`, `GHA`, `TS`), natural-language proxies (`base class hierarchy`, `exceptions`), and cross-scope aliases (`Redis` in both `domain:code` and `domain:writing`). BM25 passes all 18; vector search fails the majority due to noise from semantically adjacent beliefs.

Scope disambiguation (8 cases). The `Redis` disambiguation cases are the most stringent: both beliefs share the alias “Redis” and the query contains only “Redis.” The discriminator is scope alone, applied as a hard filter after search scoring. This property cannot be achieved by similarity search alone.

Table 2: Evaluation suite case distribution by category.

Category	Case count
Alias resolution	18
Scope disambiguation	8
Supersession chain exclusion	3
Fuzzy matching and prefix guards	8
Cross-user isolation	2
Cold start behavior	1
Budget eviction and capacity	3
Ranking stability	3
Persona prelude content	4
Type routing and open questions	4
Design boundary cases	4
Session-level noise isolation	12
Total	72

Supersession chain exclusion (3 cases). The three-hop chain (TSLint superseded by ESLint superseded by Biome) tests the filter at depth. A query containing both `ESLint` and `TSLint` must surface neither superseded belief. The current terminal belief (Biome) surfaces via the pinned facts tier.

Fuzzy matching (8 cases). The prefix guard blocks `mango` (intended: `Mongo`) because `ma` does not match `mo`. Transpositions are treated as edit distance 1, so `typscript` matches `TypeScript`. Both behaviors are documented as intentional design trade-offs.

Counter-signal retrieval. The `negative-guidance-mongoose d` case verifies the counter-signal property introduced in Section 5: the query “suggest a good Mongoose schema for my user model” retrieves `b-mongo-raw-driver` via the `moongoose` alias with score 9.29.

Cross-user isolation and cold start (3 cases). A belief belonging to a second user (Haskell preference, pinned, high confidence) must never surface for the primary test user regardless of query similarity. A brand-new user with zero seeded beliefs returns fully empty context at every tier without error.

6.3 Session-Level Noise Isolation

The 60-case static suite evaluates single-query retrieval properties against a static seed corpus. The 12-case session evaluation suite tests a single orthogonal property: whether beliefs extracted during drift turns contaminate retrieval on unrelated subsequent turns. It is not a general retrieval quality extension of the static suite. The primary case is a 10-turn session in which Redis cache performance is established at turn 0, followed by 8 substantive drift turns across Kubernetes, React, GitHub Actions, error handling, functional pipelines, and Biome configuration, followed by an implicit return to the Redis topic at turn 9 with the query “Ok circling back to what we were talking about at the start...” A noise check assertion verifies that beliefs seeded by drift turns (Kubernetes, CI, React, composition, error handling, functional pipeline) do not contaminate retrieval on the re-entry turns.

The session evaluation introduces a *drift score* metric: the fraction of retrieved non-pinned beliefs

that originate from drift-turn topics rather than the re-entry topic. A drift score of 0 indicates perfect noise isolation; a drift score approaching 1 indicates complete contamination by accumulated session context.

Table 3: Session-level noise isolation results. Drift score on turns 9–10 (noise-critical re-entry turns) and the cross-session formative case.

Turn	BM25 drift	Vector drift	Noise beliefs surfaced
Turn 9 (implicit re-entry)	0.0	0.43	6/7
Turn 10 (explicit re-entry)	0.0	0.50	7/7
Cross-session formative	0.0	0.07	1/1

Figure 2 shows the turn-by-turn drift scores. BM25 maintains 0.0 throughout: at turn 9 the implicit continuation query matches no indexed aliases and BM25 correctly returns nothing; at turn 10 the `allkeys-lru` alias match produces a score of 10.26 on `b-redis-code` alone.

The vector backend fails all 12 session turns. On turn 9 it surfaces 11 non-pinned beliefs including 6 of 7 noise beliefs (drift score 0.43); on turn 10, where the query explicitly names `allkeys-lru`, `b-redis-code` ranks first (0.853) but all 7 noise beliefs are still retrieved (drift score 0.50).

The session results demonstrate that the precision gap observed in static evaluation is not merely preserved but amplified under multi-turn accumulation pressure. Each drift turn adds semantic mass from a different topic to the session’s token footprint. For embedding-based retrieval, this accumulated mass creates broader semantic overlap with the belief corpus on every subsequent turn. For BM25 with alias-weighted search, accumulated turns are irrelevant because retrieval operates on query-term matches against indexed aliases, not on accumulated session semantics.

7 Discussion

7.1 Memory as State, Not Search

The central argument of this paper is that cross-session LLM memory is a state management problem, not a search problem. Beliefs are not retrieved because they match a query; they are active because the user’s context requires them.

A search-based memory system fails silently: when a relevant belief is not retrieved, the model generates a response without that context and the user does not know what was missing. A state-based memory system fails explicitly: a belief can be pinned, its injection verified, and the audit trail inspected to see when it was last reinforced and what session produced it.

StructMemEval [8] provides the sharpest independent formulation of this distinction: tasks requiring memory organization, state tracking, and hierarchical structure are reliably solved by structured memory agents but not by retrieval-augmented systems, even when the retrieval budget is increased. Memori’s framing converges from an engineering direction: “memory in LLM systems is not simply a storage problem, but a structuring problem” [1]. Tenure’s contribution relative to both is a formal answer to what that structure must contain: typed epistemic status, structural scope isolation, versioned supersession, a `why_it_matters` field that converts facts to instructions, and precision-first retrieval designed for named entity resolution within bounded vocabulary contexts, demonstrated

on a single-user corpus and architecturally extensible to team-level stores where shared terminology produces the same structural property.

The mechanism and failure mode are documented in Sections 4 and 6.

User-controlled scope makes the state boundary explicit and auditable. Precision-first retrieval and hard scope isolation together provide a structural guarantee: the right beliefs surface, and only within the boundaries the user has authorized. In explicit scope mode this extends to a data governance property relevant to team deployments: a session in `project:client-a` cannot surface beliefs from `project:client-b` regardless of semantic proximity, because scope is a hard filter applied before retrieval scoring. A user moving between contexts mid-session receives an immediate belief context pivot with no residual injection from the previous scope. These properties are architectural consequences of treating memory as typed state rather than a search index, and no surveyed system provides an equivalent guarantee.

7.2 Context Rot and Its Architectural Answer

Context rot is the failure mode that any persistent memory system must address. Tenure addresses it through three mechanisms. Supersession marks old beliefs as permanently non-injectable when replaced and preserves the chain for audit. Compaction continuously merges overlapping beliefs into canonical form, preventing accumulation of redundant and potentially contradictory beliefs. User control makes every belief visible, editable, and auditable.

Supersession and scope isolation are the architectural answer: superseded beliefs cannot propagate because they are never injected, and scope isolation ensures beliefs from one domain cannot contaminate another.

8 Conclusion

We have presented Tenure, a local-first belief architecture for persistent cross-session context in LLM interactions. The central claim is that cross-session LLM memory is a state management problem, not a search problem, and that similarity search is the wrong retrieval paradigm for named entity resolution within bounded vocabulary contexts, of which a single user is the simplest instance.

Together, the four contributions form a single coherent answer to the re-orientation tax: a schema that makes belief state auditable, a retrieval design that makes injection precise, a compaction architecture that keeps the store clean over time, and a benchmark that makes all three properties measurable by any comparison system. The quantitative anchor is precision: on cases with active retrieval assertions, cosine similarity achieves mean precision of 0.12 (one correct belief returned alongside nine noise beliefs on average) while passing 8/72 retrieval cases compared to BM25’s 72/72. Under multi-turn topic drift the gap widens: the vector backend produces drift scores of 0.43–0.50 on noise-critical turns where BM25 maintains 0.0 across all session turns. The eval suite is one reusable artifact; every comparison system can be measured against the same 72 cases. The other reusable artifact is the system itself: Tenure ships as a Docker container, runs on localhost, and produces the results described in this paper from the first session.

The eval suite provides 72 cases against which any comparison system can be measured directly. Tenure itself ships as a Docker container and produces the results described here from the first session.

The re-orientation tax motivating the system remains an observational claim rather than a measured one. Retrieval precision is a necessary precondition for response quality improvement, not a sufficient one; that measurement is the open question this work makes tractable.

References

- [1] Borro, L. C., Macarini, L. A. B., Tindall, G., Montero, M., and Struck, A. B. (2026). Memori: A persistent memory layer for efficient, context-aware LLM agents. *arXiv preprint arXiv:2603.19935*.
- [2] Chhikara, P., Khant, D., Aryan, S., Singh, T., and Yadav, D. (2025). Mem0: Building production-ready AI agents with scalable long-term memory. In *Proceedings of the 27th European Conference on Artificial Intelligence (ECAI 2025)*. arXiv:2504.19413.
- [3] Tenure (2025). Tenure: Structured belief state for persistent LLM context. github.com/jeffreyflynt/tenure.
- [4] Mihalcea, R. and Csomai, A. (2007). Wikify!: Linking documents to encyclopedic knowledge. In *Proceedings of the Sixteenth ACM Conference on Information and Knowledge Management, CIKM '07*, pages 233–242.
- [5] OpenAI (2023). Memory and new controls for ChatGPT. Retrieved from <https://openai.com/blog/memory-and-new-controls-for-chatgpt> (accessed May 2026).
- [6] Rao, A. S. and Georgeff, M. P. (1995). BDI agents: From theory to practice. In *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*, pages 312–319.
- [7] Robertson, S. E. and Walker, S. (1994). Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval. In *Proceedings of the 17th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR '94*, pages 232–241.
- [8] Shutova, A., Olenina, A., Vinogradov, I., and Sinitin, A. (2026). Evaluating memory structure in LLM agents. *arXiv preprint arXiv:2602.11243*.
- [9] Tang, Z., Yu, X., Xiao, Z., Wen, Z., Li, Z., Zhou, J., Wang, H., Wang, H., Huang, H., Deng, D., Sun, F., and Zhang, Q. (2026). Mnemis: Dual-route retrieval on hierarchical graphs for long-term LLM memory. *arXiv preprint arXiv:2602.15313*.
- [10] Zhang, Z., Dai, Q., Bo, X., Ma, C., Li, R., Chen, X., Zhu, J., Dong, Z., and Wen, J.-R. (2025). A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6), Article 155. <https://doi.org/10.1145/3748302>.
- [11] Xu, W., Liang, Z., Mei, K., Gao, H., Tan, J., and Zhang, Y. (2025). A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*.
- [12] Ye, Z., Huang, J., Chen, W., and Zhang, Y. (2026). H-Mem: Hybrid multi-dimensional memory management for long-context conversational agents. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL 2026)*, pages 7756–7775.
- [13] Barlow, M. (2013). Individual usage: A corpus-based study of idiolects. *International Journal of Corpus Linguistics*, 18(4), 443–478.

- [14] Hoey, M. (2005). *Lexical Priming: A New Theory of Words and Language*. Routledge.
- [15] Wright, D. (2018). Idiolect. In M. Aronoff (Ed.), *Oxford Research Encyclopedia of Linguistics*. Oxford University Press.

A Belief Schema Reference

Table 4: Belief field reference.

Field	Type	Description
<code>_id</code>	string	UUID
<code>user_id</code>	string	Owner identifier
<code>type</code>	enum	preference, decision, entity, open_question, relation
<code>subtype</code>	enum	expertise, style, null
<code>canonical_name</code>	string	Snake_case primary identifier
<code>aliases</code>	string[]	Alternative terms including counter-signals; 3–5 at extraction, hard ceiling 25 after enrichment
<code>content</code>	string	The belief statement
<code>why_it_matters</code>	string	Imperative: what future responses this shapes
<code>scope</code>	string[]	Scope labels: user:universal, domain:X, or project:X
<code>epistemic_status</code>	enum	active, inferred, exploratory, superseded
<code>confidence</code>	float	[0, 1]
<code>pinned</code>	bool	Always inject regardless of query
<code>user_edited</code>	bool	User manually authored or corrected
<code>superseded_by</code>	string?	ID of replacement belief
<code>resolved_at</code>	date?	Timestamp of open question resolution
<code>reinforcement_count</code>	int	Number of reinforcement events
<code>last_reinforced_at</code>	date	Timestamp of most recent reinforcement
<code>provenance</code>	object	session_id, turn_id, extracted_at, source_model
<code>change_log</code>	object[]	Append-only mutation history
<code>expertise_domain</code>	string?	Hierarchical domain, e.g. javascript/react
<code>expertise_depth</code>	enum?	learning, working, deep, expert
<code>expertise_evidence_count</code>	int?	Source belief count for synthesis

Fields reflect the schema present in the Section 6 evaluation (v1.0, May 2026). The authoritative and current field reference is maintained at github.com/jeffreyflynt/tenure.